

Documentación Técnica - Patrones GoF

Tabla de los 23 Patrones GoF

Nombre Patrón	Tipo o Categoría	Problema que Resuelve	Diagrama de Clases	Casos de Uso	Ventajas / Desventajas	Patrones Relacionados
Singleton	Creacional	Garantizar que una clase tenga una única instancia y proporcionar un punto global de acceso a ella.	Client → Singleton → instance	Conexión única a datos.	+ Control centralizado. - Pruebas unitarias más difíciles.	Factory Method
Factory Method	Creacional	Crear objetos sin conocer su clase.	Creator → factoryMethod () → Product	Crear agentes Phone/Chat/Email.	+ Bajo acoplamiento. - Incrementa la cantidad de clases.	Abstract Factory
Abstract Factory	Creacional	Crear familias de objetos relacionados sin especificar sus clases concretas.	AbstractFactory → Products	Creación de Familias de componentes.	+ Consistencia entre productos. - Aumenta la complejidad del diseño.	Factory Method
Builder	Creacional	Construcción de objetos complejos paso a paso.	Director → Builder → Product	Construcción de agentes.	+ Flexible. - Más código.	Abstract Factory
Prototype	Creacional	Clonar objetos.	Prototype → clone()	Duplicar configuraciones o plantillas.	+ Creación rápida de objetos. - Clonado complejo.	Builder
Adapter	Estructural	Permitir que clases con interfaces incompatibles trabajen juntas.	Client → Target ← Adapter → Adaptee	Integración de sistemas.	+ Reutilización de código - Capa extra de abstracción	Facade
Bridge	Estructural	Separar abstracción e implementación.	Abstraction → Implementor	Canales y servicios.	+ Facilita la extensión independiente. - Incrementa la complejidad inicial.	Adapter

Composite	Estructural	Manejar grupos y objetos igual.	Component → Leaf/Composite	Organización jerárquica de equipos o grupos.	+ Simplifica estructuras jerárquicas. - Puede dificultar Restricciones.	Decorator
Decorator	Estructural	Agregar funcionalidades a objetos dinámicamente.	Component ← Decorator	Agregar métricas o comportamientos adicionales.	+ Extensible. Puede generar muchos objetos pequeños.-	Proxy
Facade	Estructural	Simplificar subsistemas complejos.	Client → Facade → Subsystems	Operaciones CRUD.	+ Simplifica el uso del sistema - Puede convertirse en una clase muy grande.	Adapter
Flyweight	Estructural	Reducir el uso de memoria compartiendo objetos similares.	Client → FlyweightFactory → Flyweight	Compartir configuraciones comunes.	+ Ahorro de memoria. - Incrementa la complejidad del diseño.	Singleton
Proxy	Estructural	Controlar acceso a un objeto mediante un intermediario.	Client → Proxy → RealSubject	Seguridad y control de acceso a recursos.	+ Protección. - Agrega una capa adicional.	Decorator
Chain of Responsibility	Comportamiento	Permitir que múltiples objetos puedan procesar una solicitud.	Handler → Successor → Successor	Validaciones encadenadas.	+ Reduce el acoplamiento. - Difícil seguir el flujo de ejecución.	Command
Command	Comportamiento	Encapsular solicitudes como objetos	Invoker → Command → Receiver	Crear y eliminar agentes mediante comandos.	+ Permite deshacer operaciones y reutilizar acciones. - Incrementa el número de clases..	Memento
Interpreter	Comportamiento	Interpretar expresiones o reglas.	AbstractExpression → TerminalExpression	Reglas de negocio.	+ Flexible para definir reglas - Afectar el Rendimiento.	Visitor
Iterator	Comportamiento	Recorrer elementos de una colección sin exponer su	Aggregate → Iterator	Recorrido de listas agentes.	+ Acceso uniforme a colecciones.	Composite

		estructura interna			- Agrega objetos adicionales.	
Mediator	Comportamiento	Centralizar comunicación.	Mediator ↔ Colleagues	Comunicación entre módulos.	+ Menor acoplamiento. - El mediador puede volverse complejo.	Observer
Memento	Comportamiento	Guardar y restaurar estados anteriores de un objeto.	Originator → Memento	Historial de cambios.	+ Permite recuperación de estados. - Consume memoria adicional.	Command
Observer	Comportamiento	Notificar cambios.	Subject → Observers	Actualización de métricas.	+ Bajo acoplamiento - Puede generar muchas notificaciones.	Mediator
State	Comportamiento	Cambiar según estado.	Context → State	Estados del agente.	+ Organiza comportamientos complejos. - Incrementa el número de clases.	Strategy
Strategy	Comportamiento	Intercambiar algoritmos.	Context → Strategy	Cálculo de KPI.	+ Facilita cambiar algoritmos. - Requiere más objetos.	State
Template Method	Comportamiento	Definir estructura de algoritmo.	AbstractClass → ConcreteClass	Procesamiento de métricas y reportes.	+ Reutilización de código - Dependencia de la Herencia.	Strategy
Visitor	Comportamiento	Agregar operaciones a una estructura sin modificar sus clases.	Visitor → Element	Generación de Reportes.	+ Facilita agregar funcionalidades. - Complejo de implementar.	Composite

1. *Builder - Construcción de Agentes*

Problema

El sistema requiere crear agentes con diferentes configuraciones, como canal de atención e identificación, evitando constructores extensos y difíciles de mantener.

Solución

AgentBuilder permite construir agentes paso a paso, configurando únicamente los atributos necesarios antes de generar el objeto final.

Clases

Agent — representa un agente del sistema de Call Center.

AgentBuilder — permite construir objetos Agent de forma gradual mediante métodos de configuración.

Ejemplo

```
agente = AgentBuilder().set_channel("CHAT").build()
```

2. *Factory Method - Creación de Agentes*

Problema

El sistema debe crear agentes para distintos canales de atención sin que el usuario conozca la clase concreta que será utilizada.

Solución

AgentFactory centraliza la creación de agentes y devuelve el tipo adecuado según el canal solicitado.

Clases

AgentFactory — responsable de crear agentes según el canal seleccionado.

ChatAgent — agente especializado en atención por chat.

PhoneAgent — agente especializado en atención telefónica.

EmailAgent — agente especializado en atención por correo electrónico.

Ejemplo

```
agente = AgentFactory.create_agent("CHAT")
```

3. Singleton - Conexión Única a Datos

Problema

Los diferentes módulos del sistema necesitan acceder a la misma conexión de datos sin crear instancias duplicadas.

Solución

DatabaseConnection implementa el patrón Singleton garantizando que exista una única instancia compartida durante toda la ejecución del sistema.

Clases

DatabaseConnection — administra una única conexión utilizada por los diferentes módulos del sistema.

Ejemplo

```
conexion1 = DatabaseConnection()  
  
conexion2 = DatabaseConnection()  
  
assert conexion1 is conexion2
```

4. Decorator - Información de Desempeño

Problema

Se requiere agregar información adicional relacionada con el desempeño de los agentes sin modificar la estructura original de la clase.

Solución

PerformanceDecorator envuelve un agente y agrega información complementaria sobre métricas e indicadores de desempeño.

Clases

Agent — componente base que representa un agente.

PerformanceDecorator — decorador que añade información de desempeño al agente.

Ejemplo

agente = PerformanceDecorator(agent)

5. Strategy - Cálculo de Indicadores KPI

Problema

El sistema necesita calcular diferentes indicadores de desempeño para los agentes del Call Center utilizando distintos métodos de cálculo.

Solución

Se implementaron estrategias independientes para calcular métricas como AHT y ACW, permitiendo utilizar diferentes algoritmos sin modificar el resto del sistema.

Clases

AHTStrategy — estrategia utilizada para calcular el indicador Average Handle Time.

ACWStrategy — estrategia utilizada para calcular el indicador After Call Work.

Ejemplo

resultado = AHTStrategy().calculate()

6. Facade - Gestión Centralizada

Problema

El usuario necesita acceder a diferentes funcionalidades relacionadas con la gestión de agentes mediante una interfaz sencilla.

Solución

CallCenterFacade centraliza las operaciones principales del sistema y simplifica la interacción con los diferentes módulos.

Clases

CallCenterFacade — proporciona una interfaz única para las funcionalidades más utilizadas del sistema.

Ejemplo

```
facade = CallCenterFacade()
```

7. Command - Operaciones sobre Agentes

Problema

Las acciones relacionadas con los agentes deben ejecutarse de forma organizada y desacoplada.

Solución

AgentCommand encapsula las operaciones que pueden realizarse sobre los agentes, permitiendo ejecutarlas de manera independiente.

Clases

AgentCommand — encapsula acciones relacionadas con la gestión de agentes.

Ejemplo

```
command = AgentCommand()
```

```
command.execute()
```